Accord — ADR Starter Pack and Repo Bootstrap Plan

Status

Draft v0.1

Purpose

This document serves two roles:

1. it captures the first set of **Architecture Decision Records (ADRs)** that should be considered foundational for Accord, and
2. it defines the **repo bootstrap plan** so implementation can begin with a clean structure and a clear first sprint.

The goal is to reduce ambiguity before coding starts.

Part I — ADR Starter Pack

How to Read These ADRs

These are lightweight starter ADRs, not final legal documents. They exist to lock early intent and prevent repeated re-debates while building.

Each ADR follows this format: - **Decision** - **Why** - **Tradeoffs** - **Implication**

ADR-001 — TypeScript-First Monorepo

Decision

Accord v1 will be implemented as a **TypeScript-first monorepo**.

Why

Accord is SDK-first, local-first, and developer-facing. TypeScript gives us: - a strong fit for protocol objects and shared schemas, - fast iteration across SDK, server, and console, - one language across most of the first implementation, - and lower coordination overhead in the early phase.

Tradeoffs

- We are not maximizing polyglot ecosystem reach in v1.
- Python-first AI developers may face a slightly higher barrier initially.
- We may need adapters or additional SDKs later.

Implication

All core packages in v1 should assume TypeScript as the primary implementation language.

ADR-002 — Local-First Before Hosted

Decision

Accord v1 will be **local-first** and will not begin as a hosted multi-tenant platform.

Why

The core value of Accord is trust, inspectability, and runtime clarity. Local-first helps by: - making the full state machine inspectable, - reducing cloud complexity, - simplifying early iteration, - and keeping trust claims honest.

Tradeoffs

- No immediate SaaS convenience.
- No multi-user or team workflows in v1.
- No cloud sync or enterprise deployment story at the start.

Implication

The first implementation must run end-to-end on a developer's machine without remote dependencies.

ADR-003 — SQLite as the First Persistent Store

Decision

Accord v1 will use **SQLite** as the primary local persistence layer.

Why

SQLite is simple, local, inspectable, and sufficient for the object relationships in v1.

It supports: - linked structured records, - easy local debugging, - low setup friction, - and stable storage for audit trails and approval state.

Tradeoffs

- Not the long-term answer for distributed deployments.
- Limited story for concurrency-heavy or cloud-native use cases.
- Migrations and abstractions still need discipline.

Implication

The data model should be designed cleanly enough that Postgres or another backend can be added later, but we should not abstract storage too aggressively in v1.

ADR-004 — Accord Is a Control Layer, Not a Memory Engine

Decision

Accord will **not** build its own memory retrieval/ranking engine in v1.

Why

The most defensible wedge is not memory persistence itself. It is: - scoped context disclosure, - action approval, - auditability, - and revocation-aware runtime control.

Trying to build memory infrastructure too would dilute the project and blur the problem statement.

Tradeoffs

- Some developers may expect Accord to store and retrieve all personal context directly.
- We will need to explain clearly how Accord complements memory systems instead of replacing them.

Implication

Any context store used by sample apps should remain secondary to the control-plane flow.

ADR-005 — Approval Is a First-Class Runtime Primitive

Decision

Accord will treat **human approval** as a first-class runtime primitive rather than a UI add-on.

Why

The project's core promise depends on being able to stop meaningful side effects until approval conditions are satisfied.

That means approval must exist as: - a protocol object, - a policy outcome, - a verification step, - and a durable audit event.

Tradeoffs

- This makes the runtime slightly more opinionated.
- Developers seeking pure autonomy may see this as friction.
- Some flows become more complex by design.

Implication

Action execution should never quietly bypass approval-required paths in the reference implementation.

ADR-006 — Conservative Default Policy

Decision

Accord v1 will ship with **conservative default policies** for meaningful writes and external side effects.

Why

Most early adopters will copy defaults and examples. Unsafe defaults would undermine the project.

Tradeoffs

- Some developers may feel the system is stricter than necessary at first.
- There may be more manual approvals during examples and local testing.

Implication

Writes, commits, pushes, publishing, and comparable actions should require approval by default in the reference policy set.

ADR-007 — Sample Apps Are Proof, Not the Product

Decision

The first sample apps will exist to **prove the protocol and SDK**, not to become flagship end-user products.

Why

Accord’s long-term value comes from being adopted as infrastructure. If sample apps drive the design too early, the protocol will become secondary.

Tradeoffs

- The demos may appear more utilitarian than flashy.
- Public perception may initially be more technical than consumer-facing.

Implication

Every example should exist to exercise one or more core control-plane behaviors clearly.

ADR-008 — Protocol and Validation Before UI Polish

Decision

The protocol package, validators, server flow, and policy boundary must be built before investing heavily in UI polish.

Why

The UI is not the hard part. The trust boundary is.

Tradeoffs

- The project may feel visually underwhelming in the early phase.
- Momentum may feel slower to outsiders who expect a demo-first approach.

Implication

Early milestones should be judged by correctness and inspectability, not aesthetics.

ADR-009 — One Strong Adapter Later, Not Many Early

Decision

Accord will not chase many ecosystem adapters in the initial build.

Why

Too many adapters early would distract from the core runtime model.

Tradeoffs

- Early interoperability breadth will be limited.
- Some developers may want immediate support for their preferred framework.

Implication

The first adapter should be chosen only after the local control-plane flow and examples already feel solid.

ADR-010 — Fail Closed Whenever Approval or Scope State Is Invalid

Decision

Accord will follow a strict **fail-closed** model whenever scope, approval, expiry, or revocation state is invalid.

Why

Fail-open behavior would directly contradict the project's purpose.

Tradeoffs

- More blocked flows during development.
- Developers must handle denial and expiry states clearly.

Implication

Executors and sample integrations must verify all required state immediately before side effects.

Part II — Repo Bootstrap Plan

1. Bootstrap Objective

The repo bootstrap phase should create a project that is: - easy to install, - easy to navigate, - consistent in tooling, - and ready for the first vertical slice.

The bootstrap goal is not "set up everything." It is "set up enough to start shipping the control plane cleanly."

---

2. Proposed Initial Repo Structure

```
accord/
  README.md
  LICENSE
  package.json
  pnpm-workspace.yaml
  tsconfig.base.json
```

```
.gitignore
.editorconfig
docs/
  00-project-thesis.md
  01-research-brief.md
  02-prd.md
  03-protocol-spec.md
  04-architecture.md
  05-threat-model.md
  06-roadmap.md
  07-adr-starter-pack.md
  adr/
packages/
  protocol/
  server/
  policy-engine/
  sdk/
  console/
examples/
  coding-agent/
  research-publisher/
tooling/
  scripts/
  config/
```

Why this shape

- `docs/` holds the product and architecture narrative.
- `packages/` holds the reusable implementation pieces.
- `examples/` proves real usage.
- `tooling/` avoids clutter in root while keeping scripts organized.

---

3. Package Responsibilities

`packages/protocol`

Should contain: - TypeScript interfaces/types - schemas - validators - enums/constants - object helper utilities

This package is the foundation and should have the fewest dependencies possible.

`packages/server`

Should contain: - HTTP server - route handlers - storage module integration - scope/proposal/approval/audit services

`packages/policy-engine`

Should contain: - local policy config model - matching/evaluation logic - helper functions for enforcement outcomes

`packages/sdk`

Should contain: - typed client for talking to server - convenience methods for common flows - receipt verification helpers - developer-facing config surface

`packages/console`

Should contain: - React UI - pending approvals view - scope view - audit view - proposal detail / manifest detail views

---

4. Root Tooling Recommendations

Package Manager

Use **pnpm** for workspace management.

Why: - fast installs, - good monorepo ergonomics, - sane workspace linking, - efficient dependency management.

Build Tooling

Keep it simple in v1: - TypeScript compiler for packages - lightweight dev server for console - basic workspace scripts from the root

Linting / Formatting

- ESLint
- Prettier

Testing

Start with: - Vitest for unit/integration tests

Why keep tooling light

The project's complexity should come from its trust model, not from build-system cleverness.

---

5. Bootstrap Tasks — Ordered

Task Group A — Root Setup

1. initialize git repo
2. create root `package.json`
3. add workspace config
4. add base TypeScript config
5. add lint/format/test base config
6. add `.gitignore`, `.editorconfig`, license

Done when

The root can install and run common workspace commands successfully.

Task Group B — Docs Import and Alignment

1. move planning docs into `docs/`
2. normalize names and numbering
3. create `docs/adr/README.md`
4. create `README.md` skeleton with:
5. what Accord is
6. why it exists
7. current status
8. workspace overview

Done when

A new visitor can understand the project shape from docs without reading implementation code.

Task Group C — Protocol Package Bootstrap

1. initialize `packages/protocol`
2. add TypeScript config
3. create folders for:
4. `types/`
5. `schemas/`
6. `validators/`
7. `constants/`

8. define initial protocol object types
9. implement schema validation
10. add fixture tests for valid/invalid payloads

Done when

Protocol objects compile, validate, and fail clearly when malformed.

Task Group D — Server Package Bootstrap

1. initialize `packages/server`
2. add minimal HTTP server
3. define route groups
4. connect protocol validators
5. add SQLite setup and migrations
6. create storage helpers for all major objects
7. add append-only audit writing on major state changes

Done when

A local server can accept and persist the core protocol objects.

Task Group E — Policy Engine Bootstrap

1. initialize `packages/policy-engine`
2. define policy config format
3. implement evaluation for scope and action proposals
4. add default conservative policy file
5. add test coverage for allow / require_approval / deny

Done when

The server can classify proposals and reject invalid execution paths.

Task Group F — SDK Bootstrap

1. initialize `packages/sdk`
2. add typed API client
3. implement first helper methods:
4. `requestScope`
5. `publishContextManifest`
6. `proposeAction`
7. `verifyReceipt`

8. add examples of raw vs convenience usage

Done when

A local developer can use the SDK without manually crafting every HTTP call.

Task Group G — Console Bootstrap

1. initialize `packages/console`
2. create basic app shell
3. add pending proposals page
4. add proposal detail page with manifest linkage
5. add active scopes page
6. add audit history page
7. add revoke and approve/deny actions

Done when

A human can inspect and act on the control-plane state visually.

Task Group H — Example App Bootstrap

1. initialize `examples/coding-agent`
2. wire to SDK + server
3. demonstrate proposal → approval → execution
4. initialize `examples/research-publisher`
5. repeat the same control-plane proof with a different action type

Done when

Two examples prove that Accord is reusable rather than app-specific.

---

6. Suggested First Sprint

The first sprint should avoid spreading effort across all packages.

Recommended Sprint Goal

Make the protocol and the first local state transitions real.

Sprint Scope

- root repo setup

- docs import
- protocol package skeleton
- server bootstrap
- SQLite schema bootstrap
- create + store:
- `ScopeRequest`
- `GrantedScope`
- `ActionProposal`
- `AuditRecord`

Sprint Proof

By the end of the first sprint, it should be possible to: - run the local server, - submit a scope request, - issue a granted scope, - create an action proposal, - and inspect the stored audit records.

That is enough for a serious start.

---

7. Suggested Second Sprint

Sprint Goal

Make approval and verification real.

Sprint Scope

- policy evaluation
- approval receipts
- expiry/revocation checks
- proposal status transitions
- fail-closed tests

Sprint Proof

A write-like action marked `require_approval` must remain blocked until a valid approval receipt exists.

---

8. Suggested Third Sprint

Sprint Goal

Make the system inspectable and pleasant enough to demo honestly.

Sprint Scope

- minimal console
- proposal detail + manifest view

- active scopes
- audit history
- first SDK ergonomics pass

Sprint Proof

A person can review a proposal, see what context influenced it, approve it, and inspect the resulting history.

---

9. Initial Root Scripts to Add

At minimum, the repo should support: - `dev` - `build` - `test` - `lint` - `format` - `typecheck`

Later we can add package-specific scripts, but these six are enough to begin.

---

10. Initial ADR Files to Create Separately

After this starter pack, the following individual ADR files should be created under `docs/adr/`:

- `001-typescript-first.md`
- `002-local-first-v1.md`
- `003-sqlite-first.md`
- `004-not-a-memory-engine.md`
- `005-approval-first-class.md`
- `006-conservative-default-policy.md`
- `007-sample-apps-are-proof.md`
- `008-protocol-before-polish.md`
- `009-few-adapters-early.md`
- `010-fail-closed-default.md`

These can start as concise files and mature as implementation proceeds.

---

11. Early Success Checklist

Before moving beyond bootstrap, these should be true: - repo installs cleanly - docs are visible and coherent - protocol package validates core objects - server persists core objects locally - audit records are being written - policy engine can classify proposals - approval flow is blocked until receipt exists

If these are not true, we are not ready to widen scope.

---

12. Closing Statement

The repo bootstrap phase should make Accord feel inevitable.

Not flashy. Not broad. Not overbuilt.

Just clear enough that a contributor can look at the repository and understand: - what the project is, - what problem it solves, - where the protocol lives, - where the runtime lives, - and what the first implementation should prove.

That is the right starting point for a project that wants to become real infrastructure.